

CHAPTER 8

QUEUES

IN THIS CHAPTER, YOU WILL:

- Learn about queues
- Examine various queue operations
- Learn how to implement a queue as an array
- Learn how to implement a queue as a linked list
- Discover queue applications
- Become aware of the STL `class queue`

This chapter discusses another important data structure, called a **queue**. The notion of a queue in computer science is the same as the notion of the queues to which you are accustomed in everyday life. There are queues of customers in a bank or in a grocery store and queues of cars waiting to pass through a tollbooth. Similarly, because a computer can send a print request faster than a printer can print, a queue of documents is often waiting to be printed at a printer. The general rule to process elements in a queue is that the customer at the front of the queue is served next and that when a new customer arrives, he or she stands at the end of the queue. That is, a queue is a First In First Out data structure.

Queues have numerous applications in computer science. Whenever a system is modeled on the First In First Out principle, queues are used. At the end of this section, we will discuss one of the most widely used applications of queues, computer simulation. First, however, we need to develop the tools necessary to implement a queue. The next few sections discuss how to design classes to implement queues as an ADT.

A queue is a set of elements of the same type in which the elements are added at one end, called the **back** or **rear**, and deleted from the other end, called the **front**. For example, consider a line of customers in a bank, wherein the customers are waiting to withdraw/deposit money or to conduct some other business. Each new customer gets in the line at the rear. Whenever a teller is ready for a new customer, the customer at the front of the line is served.

The rear of the queue is accessed whenever a new element is added to the queue, and the front of the queue is accessed whenever an element is deleted from the queue. As in a stack, the middle elements of the queue are inaccessible, even if the queue elements are stored in an array.

Queue: A data structure in which the elements are added at one end, called the rear, and deleted from the other end, called the front; a First In First Out (FIFO) data structure.

Queue Operations

From the definition of queues, we see that the two key operations are add and delete. We call the add operation **`addQueue`** and the delete operation **`deleteQueue`**. Because elements can be neither deleted from an empty queue nor added to a full queue, we need two more operations to successfully implement the **`addQueue`** and **`deleteQueue`** operations: **`isEmptyQueue`** (checks whether the queue is empty) and **`isFullQueue`** (checks whether a queue is full).

We also need an operation, **`initializeQueue`**, to initialize the queue to an empty state. Moreover, to retrieve the first and last elements of the queue, we include the operations **`front`** and **`back`** as described in the following list. Some of the queue operations are as follows:

- **`initializeQueue`**—Initializes the queue to an empty state.
- **`isEmptyQueue`**—Determines whether the queue is empty. If the queue is empty, it returns the value `true`; otherwise, it returns the value `false`.

- `isFullQueue`—Determines whether the queue is full. If the queue is full, it returns the value `true`; otherwise, it returns the value `false`.
- `front`—Returns the front, that is, the first element of the queue. Prior to this operation, the queue must exist and must not be empty.
- `back`—Returns the last element of the queue. Prior to this operation, the queue must exist and must not be empty.
- **`addQueue`**—Adds a new element to the rear of the queue. Prior to this operation, the queue must exist and must not be full.
- **`deleteQueue`**—Removes the front element from the queue. Prior to this operation, the queue must exist and must not be empty.

As in the case of a stack, a queue can be stored in an array or in a linked structure. We will consider both implementations. Because elements are added at one end and removed from the other end, we need two pointers to keep track of the front and rear of the queue, called **`queueFront`** and **`queueRear`**.

The following abstract `class queueADT` defines these operations as an ADT:

```
//*************************************************************
// Author: D.S. Malik
//
// This class specifies the basic operations on a queue.
//*************************************************************

template <class Type>
class queueADT
{
public:
    virtual bool isEmptyQueue() const = 0;
      //Function to determine whether the queue is empty.
      //Postcondition: Returns true if the queue is empty,
      //    otherwise returns false.

    virtual bool isFullQueue() const = 0;
      //Function to determine whether the queue is full.
      //Postcondition: Returns true if the queue is full,
      //    otherwise returns false.

    virtual void initializeQueue() = 0;
      //Function to initialize the queue to an empty state.
      //Postcondition: The queue is empty.

    virtual Type front() const = 0;
      //Function to return the first element of the queue.
      //Precondition: The queue exists and is not empty.
      //Postcondition: If the queue is empty, the program
      //    terminates; otherwise, the first element of the queue
      //    is returned.
```

```
    virtual Type back() const = 0;
      //Function to return the last element of the queue.
      //Precondition: The queue exists and is not empty.
      //Postcondition: If the queue is empty, the program
      //    terminates; otherwise, the last element of the queue
      //    is returned.

    virtual void addQueue(const Type& queueElement) = 0;
      //Function to add queueElement to the queue.
      //Precondition: The queue exists and is not full.
      //Postcondition: The queue is changed and queueElement is
      //    added to the queue.

    virtual void deleteQueue() = 0;
      //Function to remove the first element of the queue.
      //Precondition: The queue exists and is not empty.
      //Postcondition: The queue is changed and the first element
      //    is removed from the queue.
};
```

We leave it as an exercise for you to draw the UML diagram of the `class queueADT`.

Implementation of Queues as Arrays

Before giving the definition of the class to implement a queue as an ADT, we need to decide how many member variables are needed to implement the queue. Of course, we need an array to store the queue elements, the variables `queueFront` and `queueRear` to keep track of the first and last elements of the queue, and the variable `maxQueueSize` to specify the maximum size of the queue. Thus, we need at least four member variables.

Before writing the algorithms to implement the queue operations, we need to decide how to use `queueFront` and `queueRear` to access the queue elements. How do `queueFront` and `queueRear` indicate that the queue is empty or full? Suppose that `queueFront` gives the index of the first element of the queue, and `queueRear` gives the index of the last element of the queue. To add an element to the queue, first we advance `queueRear` to the next array position and then add the element to the position that `queueRear` is pointing to. To delete an element from the queue, first we retrieve the element that `queueFront` is pointing to and then advance `queueFront` to the next element of the queue. Thus, `queueFront` changes after each `deleteQueue` operation and `queueRear` changes after each `addQueue` operation.

Let us see what happens when `queueFront` changes after a `deleteQueue` operation and `queueRear` changes after an `addQueue` operation. Assume that the array to hold the queue elements is of size 100.

Initially, the queue is empty. After the operation:

```
addQueue(Queue,'A');
```

the array is as shown in Figure 8-1.

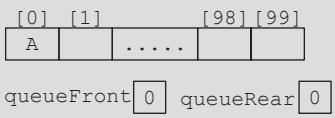


FIGURE 8-1 Queue after the first `addQueue` operation

After two more **`addQueue`** operations:

```
addQueue(Queue,'B');
addQueue(Queue,'C');
```

the array is as shown in Figure 8-2.

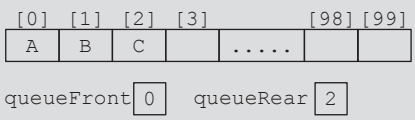


FIGURE 8-2 Queue after two more `addQueue` operations

Now consider the **`deleteQueue`** operation:

```
deleteQueue();
```

After this operation, the array containing the queue is as shown in Figure 8-3.

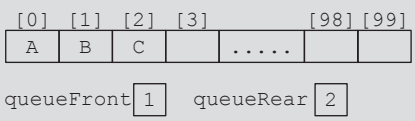


FIGURE 8-3 Queue after the `deleteQueue` operation

Will this queue design work? Suppose **`A`** stands for adding (that is, **`addQueue`**) an element to the queue, and **`D`** stands for deleting (that is, **`deleteQueue`**) an element from the queue. Consider the following sequence of operations:

```
AAADADADADADADADA...
```

This sequence of operations would eventually set the index **queueRear** to point to the last array position, giving the impression that the queue is full. However, the queue has only two or three elements and the front of the array is empty. (See Figure 8-4.)

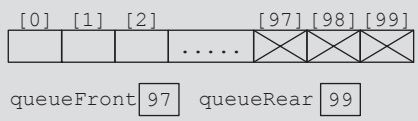


FIGURE 8-4 Queue after the sequence of operations AAADADADADADA...

One solution to this problem is that when the queue overflows to the rear (that is, **queueRear** points to the last array position), we can check the value of the index **queueFront**. If the value of **queueFront** indicates that there is room in the front of the array, then when **queueRear** gets to the last array position, we can slide all of the queue elements toward the first array position. This solution is good if the queue size is very small; otherwise, the program might execute more slowly.

Another solution to this problem is to assume that the array is circular—that is, the first array position immediately follows the last array position. (See Figure 8-5.)

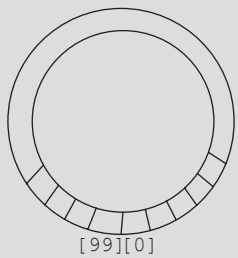


FIGURE 8-5 Circular queue

We will consider the array containing the queue to be circular, although we will draw the figures of the array holding the queue elements as before.

Suppose that we have the queue as shown in Figure 8-6(a).

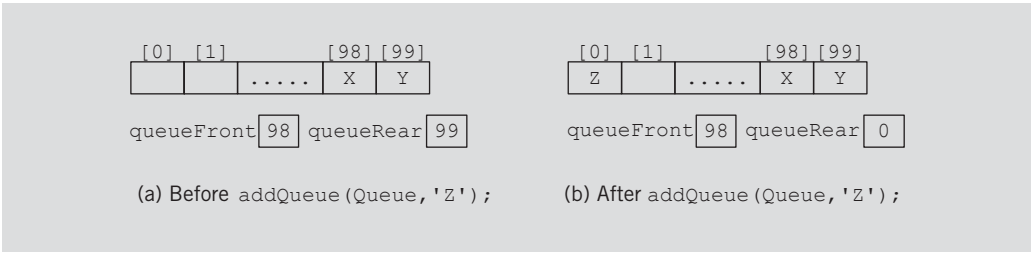


FIGURE 8-6 Queue before and after the add operation

After the operation `addQueue(Queue,'Z');`, the queue is as shown in Figure 8-6(b).

Because the array containing the queue is circular, we can use the following statement to advance `queueRear` (`queueFront`) to the next array position:

```
queueRear = (queueRear + 1) % maxQueueSize;
```

If `queueRear < maxQueueSize - 1`, then `queueRear + 1 <= maxQueueSize - 1`, so `(queueRear + 1) % maxQueueSize = queueRear + 1`. If `queueRear == maxQueueSize - 1` (that is, `queueRear` points to the last array position), `queueRear + 1 == maxQueueSize`, so `(queueRear + 1) % maxQueueSize = 0`. In this case, `queueRear` will be set to `0`, which is the first array position.

This queue design seems to work well. Before we write the algorithms to implement the queue operations, consider the following two cases.

Case 1: Suppose that after certain operations, the array containing the queue is as shown in Figure 8-7(a).

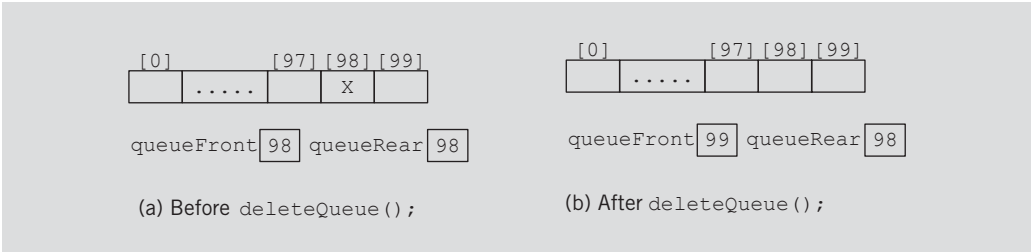


FIGURE 8-7 Queue before and after the delete operation

After the operation `deleteQueue();`, the resulting array is as shown in Figure 8-7(b).

Case 2: Let us now consider the queue shown in Figure 8-8(a).

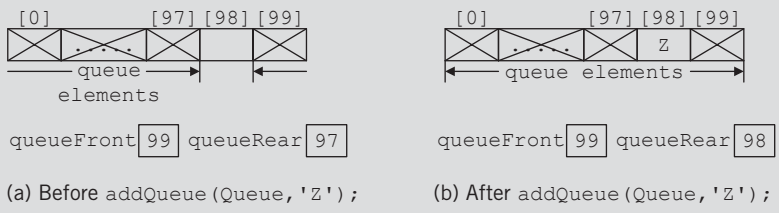


FIGURE 8-8 Queue before and after the add operation

After the operation `addQueue(Queue,'Z');`, the resulting array is as shown in Figure 8-8(b).

The arrays in Figures 8-7(b) and 8-8(b) have identical values for `queueFront` and `queueRear`. However, the resulting array in Figure 8-7(b) represents an empty queue, whereas the resulting array in Figure 8-8(b) represents a full queue. This latest queue design has brought up another problem of distinguishing between an empty and a full queue.

This problem has several solutions. One solution is to keep a count. In addition to the member variables `queueFront` and `queueRear`, we need another variable, `count`, to implement the queue. The value of `count` is incremented whenever a new element is added to the queue, and it is decremented whenever an element is removed from the queue. In this case, the function `initializeQueue` initializes `count` to `0`. This solution is very useful if the user of the queue frequently needs to know the number of elements in the queue.

Another solution is to let `queueFront` indicate the index of the array position *preceding* the first element of the queue, rather than the index of the (actual) first element itself. In this case, assuming `queueRear` still indicates the index of the last element in the queue, the queue is empty if `queueFront == queueRear`. In this solution, the slot indicated by the index `queueFront` (that is, the slot preceding the first true element) is reserved. The queue will be full if the next available space is the special reserved slot indicated by `queueFront`. Finally, because the array position indicated by `queueFront` is to be kept empty, if the array size is, say, `100`, then 99 elements can be stored in the queue. (See Figure 8-9.)

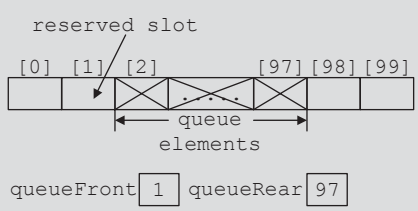


FIGURE 8-9 Array to store the queue elements with a reserved slot

Let us implement the queue using the first solution. That is, we use the variable `count` to indicate whether the queue is empty or full.

The following class implements the functions of the abstract `class queueADT`. Because arrays can be allocated dynamically, we will leave it for the user to specify the size of the array to implement the queue. The default size of the array is `100`.

```
//*************************************************************
// Author: D.S. Malik
//
// This class specifies the basic operation on a queue as an
// array.
//*************************************************************

template <class Type>
class queueType: public queueADT<Type>
{
public:
    const queueType<Type>& operator=(const queueType<Type>&);
      //Overload the assignment operator.

    bool isEmptyQueue() const;
      //Function to determine whether the queue is empty.
      //Postcondition: Returns true if the queue is empty,
      //    otherwise returns false.

    bool isFullQueue() const;
      //Function to determine whether the queue is full.
      //Postcondition: Returns true if the queue is full,
      //    otherwise returns false.

    void initializeQueue();
      //Function to initialize the queue to an empty state.
      //Postcondition: The queue is empty.

    Type front() const;
      //Function to return the first element of the queue.
      //Precondition: The queue exists and is not empty.
      //Postcondition: If the queue is empty, the program
      //    terminates; otherwise, the first element of the
      //    queue is returned.

    Type back() const;
      //Function to return the last element of the queue.
      //Precondition: The queue exists and is not empty.
      //Postcondition: If the queue is empty, the program
      //    terminates; otherwise, the last element of the queue
      //    is returned.

    void addQueue(const Type& queueElement);
      //Function to add queueElement to the queue.
      //Precondition: The queue exists and is not full.
```

```
      //Postcondition: The queue is changed and queueElement is
      //    added to the queue.

    void deleteQueue();
      //Function to remove the first element of the queue.
      //Precondition: The queue exists and is not empty.
      //Postcondition: The queue is changed and the first element
      //    is removed from the queue.

    queueType(int queueSize = 100);
      //Constructor

    queueType(const queueType<Type>& otherQueue);
      //Copy constructor

    ~queueType();
      //Destructor

private:
    int maxQueueSize; //variable to store the maximum queue size
    int count;        //variable to store the number of
                      //elements in the queue
    int queueFront;   //variable to point to the first
                      //element of the queue
    int queueRear;    //variable to point to the last
                      //element of the queue
    Type *list;       //pointer to the array that holds
                      //the queue elements
};
```

We leave the UML diagram of the `class queueType` as an exercise for you. (See Exercise 15 at the end of this chapter.)

Next, we consider the implementation of the queue operations.

Empty Queue and Full Queue

As discussed earlier, the queue is empty if `count == 0`, and the queue is full if `count == maxQueueSize`. So the functions to implement these operations are as follows:

```
template <class Type>
bool queueType<Type>::isEmptyQueue() const
{
    return (count == 0);
} //end isEmptyQueue

template <class Type>
bool queueType<Type>::isFullQueue() const
{
    return (count == maxQueueSize);
} //end isFullQueue
```

Initialize Queue

This operation initializes a queue to an empty state. The first element is added at the first array position. Therefore, we initialize `queueFront` to `0`, `queueRear` to `maxQueueSize - 1`, and `count` to `0`. See Figure 8-10.

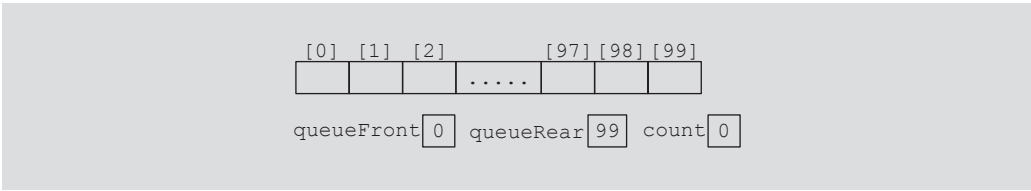


FIGURE 8-10 Empty queue

The definition of the function `initializeQueue` is as follows:

```
template <class Type>
void queueType<Type>::initializeQueue()
{
    queueFront = 0;
    queueRear = maxQueueSize - 1;
    count = 0;
} //end initializeQueue
```

Front

This operation returns the first element of the queue. If the queue is nonempty, the element of the queue indicated by the index `queueFront` is returned; otherwise, the program terminates.

```
template <class Type>
Type queueType<Type>::front() const
{
    assert(!isEmptyQueue());
    return list[queueFront];
} //end front
```

Back

This operation returns the last element of the queue. If the queue is nonempty, the element of the queue indicated by the index `queueRear` is returned; otherwise, the program terminates.

```
template <class Type>
Type queueType<Type>::back() const
{
    assert(!isEmptyQueue());
    return list[queueRear];
} //end back
```

Add Queue

Next, we implement the `addQueue` operation. Because `queueRear` points to the last element of the queue, to add a new element to the queue, we first advance `queueRear` to the next array position, and then add the new element to the array position indicated by `queueRear`. We also increment `count` by 1. So the function `addQueue` is as follows:

```
template <class Type>
void queueType<Type>::addQueue(const Type& newElement)
{
    if (!isFullQueue())
    {
        queueRear = (queueRear + 1) % maxQueueSize; //use the
                            //mod operator to advance queueRear
                            //because the array is circular
        count++;
        list[queueRear] = newElement;
    }
    else
        cout << "Cannot add to a full queue." << endl;
} //end addQueue
```

Delete Queue

To implement the `deleteQueue` operation, we access the index `queueFront`. Because `queueFront` points to the array position containing the first element of the queue, to remove the first queue element, we decrement `count` by 1 and advance `queueFront` to the next queue element. So the function `deleteQueue` is as follows:

```
template <class Type>
void queueType<Type>::deleteQueue()
{
    if (!isEmptyQueue())
    {
        count--;
        queueFront = (queueFront + 1) % maxQueueSize; //use the
                        //mod operator to advance queueFront
                        //because the array is circular
    }
    else
        cout << "Cannot remove from an empty queue" << endl;
} //end deleteQueue
```

Constructors and Destructors

To complete the implementation of the queue operations, we next consider the implementation of the constructor and the destructor. The constructor gets the `maxQueueSize` from the user, sets the variable `maxQueueSize` to the value specified by the user, and creates an array of size `maxQueueSize`. If the user does not specify the queue size, the constructor uses the default value, which is 100, to create an array of size 100. The

constructor also initializes `queueFront` and `queueRear` to indicate that the queue is empty. The definition of the function to implement the constructor is as follows:

```
template <class Type>
queueType<Type>::queueType(int queueSize)
{
    if (queueSize <= 0)
    {
        cout << "Size of the array to hold the queue must "
             << "be positive." << endl;
        cout << "Creating an array of size 100." << endl;

        maxQueueSize = 100;
    }
    else
        maxQueueSize = queueSize;   //set maxQueueSize to
                                    //queueSize

    queueFront = 0;                 //initialize queueFront
    queueRear = maxQueueSize - 1;   //initialize queueRear
    count = 0;
    list = new Type[maxQueueSize];  //create the array to
                                    //hold the queue elements
} //end constructor
```

The array to store the queue elements is created dynamically. Therefore, when the queue object goes out of scope, the destructor simply deallocates the memory occupied by the array that stores the queue elements. The definition of the function to implement the destructor is as follows:

```
template <class Type>
queueType<Type>::~queueType()
{
    delete [] list;
}
```

The implementation of the copy constructor and overloading the assignment operator are left as exercises for you, (see Programming Exercise 1 at the end of this chapter). (The definitions of these functions are similar to those discussed for array-based lists and stacks.)

Linked Implementation of Queues

Because the size of the array to store the queue elements is fixed, only a finite number of queue elements can be stored in the array. Also, the array implementation of the queue requires the array to be treated in a special way together with the values of the indices `queueFront` and `queueRear`. The linked implementation of a queue simplifies many of the special cases of the array implementation and, because the memory to store a queue element is allocated dynamically, the queue is never full. This section discusses the linked implementation of a queue.

Because elements are added at one end, `queueRear`, and removed from the other end, `queueFront`, we need to know the front of the queue and the rear of the queue. Thus, we need two pointers, **`queueFront`** and **`queueRear`**, to maintain the queue. The following class implements the functions of the abstract `class queueADT`:

```
//*************************************************************
// Author: D.S. Malik
//
// This class specifies the basic operations on a queue as a
// linked list.
//*************************************************************

//Definition of the node
template <class Type>
struct nodeType
{
    Type info;
    nodeType<Type> *link;
};

template <class Type>
class linkedQueueType: public queueADT<Type>
{
public:
    const linkedQueueType<Type>& operator=
                    (const linkedQueueType<Type>&);
      //Overload the assignment operator.

    bool isEmptyQueue() const;
      //Function to determine whether the queue is empty.
      //Postcondition: Returns true if the queue is empty,
      //    otherwise returns false.

    bool isFullQueue() const;
      //Function to determine whether the queue is full.
      //Postcondition: Returns true if the queue is full,
      //    otherwise returns false.

    void initializeQueue();
      //Function to initialize the queue to an empty state.
      //Postcondition: queueFront = NULL; queueRear = NULL

    Type front() const;
      //Function to return the first element of the queue.
      //Precondition: The queue exists and is not empty.
      //Postcondition: If the queue is empty, the program
      //    terminates; otherwise, the first element of the
      //    queue is returned.

    Type back() const;
      //Function to return the last element of the queue.
      //Precondition: The queue exists and is not empty.
```

```
      //Postcondition: If the queue is empty, the program
      //    terminates; otherwise, the last element of the
      //    queue is returned.

    void addQueue(const Type& queueElement);
      //Function to add queueElement to the queue.
      //Precondition: The queue exists and is not full.
      //Postcondition: The queue is changed and queueElement is
      //    added to the queue.

    void deleteQueue();
      //Function to remove the first element of the queue.
      //Precondition: The queue exists and is not empty.
      //Postcondition: The queue is changed and the first element
      //    is removed from the queue.

    linkedQueueType();
      //Default constructor

    linkedQueueType(const linkedQueueType<Type>& otherQueue);
      //Copy constructor

    ~linkedQueueType();
      //Destructor

private:
    nodeType<Type> *queueFront; //pointer to the front of the queue
    nodeType<Type> *queueRear;  //pointer to the rear of the queue
};
```

The UML diagram of the `class linkedQueueType` is left as an exercise for you. (See Exercise 16 at the end of this chapter.)

Next, we write the definitions of the functions of the `class linkedQueueType`.

Empty and Full Queue

The queue is empty if `queueFront` is `NULL`. Memory to store the queue elements is allocated dynamically. Therefore, the queue is never full and so the function to implement the `isFullQueue` operation returns the value `false`. (The queue is full only if the program runs out of memory.)

```
template <class Type>
bool linkedQueueType<Type>::isEmptyQueue() const
{
    return(queueFront == NULL);
} //end

template <class Type>
bool linkedQueueType<Type>::isFullQueue() const
{
    return false;
} //end isFullQueue
```

Note that in reality, in the linked implementation of queues, the function `isFullQueue` does not apply because logically the queue is never full. However, you must provide its definition because it is included as an abstract function in the parent `class queueADT`.

Initialize Queue

The operation `initializeQueue` initializes the queue to an empty state. The queue is empty if there are no elements in the queue. Note that the constructor initializes the queue when the queue object is declared. So this operation must remove all the elements, if any, from the queue. Therefore, this operation traverses the list containing the queue starting at the first node, and it deallocates the memory occupied by the queue elements. The definition of this function is as follows:

```
template <class Type>
void linkedQueueType<Type>::initializeQueue()
{
    nodeType<Type> *temp;

    while (queueFront!= NULL)  //while there are elements left
                               //in the queue
    {
        temp = queueFront;  //set temp to point to the current node
        queueFront = queueFront->link;  //advance first to
                                        //the next node
        delete temp;    //deallocate memory occupied by temp
    }

    queueRear = NULL;  //set rear to NULL
} //end initializeQueue
```

`addQueue`, `front`, `back`, and `deleteQueue` Operations

The `addQueue` operation adds a new element at the end of the queue. To implement this operation, we access the pointer `queueRear`.

If the queue is nonempty, the operation `front` returns the first element of the queue and so the element of the queue indicated by the pointer `queueFront` is returned. If the queue is empty, the function `front` terminates the program.

If the queue is nonempty, the operation `back` returns the last element of the queue and so the element of the queue indicated by the pointer `queueRear` is returned. If the queue is empty, the function `back` terminates the program. Similarly, if the queue is nonempty, the operation `deleteQueue` removes the first element of the queue, and so we access the pointer `queueFront`.

The definitions of the functions to implement these operations are as follows:

```
template <class Type>
void linkedQueueType<Type>::addQueue(const Type& newElement)
```

```
{
    nodeType<Type> *newNode;

    newNode = new nodeType<Type>;   //create the node

    newNode->info = newElement; //store the info
    newNode->link = NULL;  //initialize the link field to NULL

    if (queueFront == NULL) //if initially the queue is empty
    {
        queueFront = newNode;
        queueRear = newNode;
    }
    else        //add newNode at the end
    {
        queueRear->link = newNode;
        queueRear = queueRear->link;
    }
}//end addQueue

template <class Type>
Type linkedQueueType<Type>::front() const
{
    assert(queueFront != NULL);
    return queueFront->info;
} //end front

template <class Type>
Type linkedQueueType<Type>::back() const
{
    assert(queueRear!= NULL);
    return queueRear->info;
} //end back

template <class Type>
void linkedQueueType<Type>::deleteQueue()
{
    nodeType<Type> *temp;

    if (!isEmptyQueue())
    {
        temp = queueFront;  //make temp point to the first node
        queueFront = queueFront->link; //advance queueFront

        delete temp;    //delete the first node

        if (queueFront == NULL) //if after deletion the
                                //queue is empty
            queueRear = NULL;   //set queueRear to NULL
    }
    else
        cout << "Cannot remove from an empty queue" << endl;
}//end deleteQueue
```

The definition of the default constructor is as follows:

```
template<class Type>
linkedQueueType<Type>::linkedQueueType()
{
    queueFront = NULL; //set front to null
    queueRear = NULL;  //set rear to null
} //end default constructor
```

When the queue object goes out of scope, the destructor destroys the queue; that is, it deallocates the memory occupied by the elements of the queue. The definition of the function to implement the destructor is similar to the definition of the function `initializeQueue`. Also, the functions to implement the copy constructor and overload the assignment operators are similar to the corresponding functions for stacks. Implementing these operations is left as an exercise for you, (see Programming Exercise 2 at the end of this chapter).

EXAMPLE 8-1

The following program tests various operations on a queue. It uses the `class linkedQueueType` to implement a queue.

```
//*****************************************************************
// Author: D.S. Malik
//
// This program illustrates how to use the class linkedQueueType
// in a program.
//*****************************************************************

#include <iostream>
#include "linkedQueue.h"

using namespace std;

int main()
{
    linkedQueueType<int> queue;
    int x, y;

    queue.initializeQueue();
    x = 4;
    y = 5;
    queue.addQueue(x);
    queue.addQueue(y);
    x = queue.front();
    queue.deleteQueue();
    queue.addQueue(x + 5);
    queue.addQueue(16);
    queue.addQueue(x);
    queue.addQueue(y - 3);

    cout << "Queue Elements: ";
```

```
    while (!queue.isEmptyQueue())
    {
        cout << queue.front() << " ";
        queue.deleteQueue();
    }

    cout << endl;

    return 0;
}
```

Sample Run:

```
Queue Elements: 5 9 16 4 2
```

Queue Derived from the `class unorderedLinkedList`

From the definitions of the functions to implement the queue operations, it is clear that the linked implementation of a queue is similar to the implementation of a linked list created in a forward manner (see Chapter 5). The `addQueue` operation is similar to the operation `insertFirst`. Likewise, the operations `initializeQueue` and `initializeList`, `isEmptyQueue`, and `isEmptyList` are similar. The `deleteQueue` operation can be implemented as before. The pointer `queueFront` is the same as the pointer `first`, and the pointer `queueRear` is the same as the pointer `last`. This correspondence suggests that we can derive the class to implement the queue from the `class linkedListType` (see Chapter 5). Note that the `class linkedListType` is an abstract and does not implement all the operations. However, the `class unorderedLinkedList` is derived from the the `class linkedListType` and provides the definitions of the abstract functions of the the `class linkedListType`. Therefore, we can derive the `class linkedQueueType` from the `class unorderedLinkedList`.

We leave it as an exercise for you to write the definition of the `class linkedQueueType` that is derived from the `class unorderedLinkedList` (see Programming Exercise 7 at the end of this chapter).

STL `class queue` (Queue Container Adapter)

The preceding sections discussed the data structure queue in detail. Because a queue is an important data structure, the Standard Template Library (STL) provides a class to implement queues in a program. The name of the class defining the queue is `queue`, and the name of the header file containing the definition of the `class queue` is `queue`. The `class queue` provided by the STL is implemented similar to the classes discussed in this chapter. Table 8-1 defines various operations supported by the queue container class.

TABLE 8-1 Operations on a `queue` object

Operation	Effect
`size`	Returns the actual number of elements in the queue.
`empty`	Returns **`true`** if the queue is empty, and **`false`** otherwise.
`push(item)`	Inserts a copy of **`item`** into the queue.
`front`	Returns the next—that is, first—element in the queue, but does not remove the element from the queue. This operation is implemented as a value-returning function.
`back`	Returns the last element in the queue, but does not remove the element from the queue. This operation is implemented as a value-returning function.
`pop`	Removes the next element in the queue.

In addition to the operations **`size`**, **`empty`**, **`push`**, **`front`**, **`back`**, and **`pop`**, the queue container class provides relational operators to compare two queues. For example, the relational operator **==** can be used to determine whether two queues are identical.

The program in Example 8-2 illustrates how to use the queue container class.

EXAMPLE 8-2

```
//*************************************************************
// Author: D.S. Malik
//
// This program illustrates how to use the STL class queue in a
// program.
//*************************************************************

#include <iostream>                                          //Line 1
#include <queue>                                             //Line 2

using namespace std;                                         //Line 3

int main()                                                   //Line 4
{                                                            //Line 5
    queue<int> intQueue;                                     //Line 6

    intQueue.push(26);                                       //Line 7
    intQueue.push(18);                                       //Line 8
    intQueue.push(50);                                       //Line 9
    intQueue.push(33);                                       //Line 10
```

```
    cout << "Line 11: The front element of intQueue: "
         << intQueue.front() << endl;                    //Line 11

    cout << "Line 12: The last element of intQueue: "
         << intQueue.back() << endl;                     //Line 12

    intQueue.pop();                                      //Line 13

    cout << "Line 14: After the pop operation, the "
         << "front element of intQueue: "
         << intQueue.front() << endl;                    //Line 14

    cout << "Line 15: intQueue elements: ";              //Line 15

    while (!intQueue.empty())                            //Line 16
    {                                                    //Line 17
        cout << intQueue.front() << " ";                 //Line 18
        intQueue.pop();                                  //Line 19
    }                                                    //Line 20

    cout << endl;                                        //Line 21

    return 0;                                            //Line 22
}                                                        //Line 23
```

Sample Run:

```
Line 11: The front element of intQueue: 26
Line 12: The last element of intQueue: 33
Line 14: After the pop operation, the front element of intQueue: 18
Line 15: intQueue elements: 18 50 33
```

The preceding output is self-explanatory. The details are left as an exercise for you.

Priority Queues

The preceding sections describe how to implement a queue in a program. The use of a queue structure ensures that the items are processed in the order they are received. For example, in a banking environment, the customers who arrive first are served first. However, there are certain situations when this First In First Out rule needs to be relaxed somewhat. In a hospital environment, patients are, usually, seen in the order they arrive. Therefore, you could use a queue to ensure that the patients are seen in the order they arrive. However, if a patient arrives with severe or life-threatening symptoms, they are treated first. In other words, these patients take priority over the patients who can wait to be seen, such as those awaiting their routine annual checkup. For another example, in a shared environment, when print requests are sent to the printer, interactive programs take priority over batch-processing programs.

There are many other situations where some priority is assigned to the customers. To implement such a data structure in a program, we use a special type of queue, called **priority queues**. In a priority queue, customers or jobs with higher priority are pushed to the front of the queue.

One way to implement a priority queue is to use an ordinary linked list, which keeps the items in order from the highest to lowest priority. However, an effective way to implement a priority queue is to use a treelike structure. In Chapter 10, we discuss a special type of sorting algorithm, called the *heapsort*, which uses a treelike structure to sort a list. After describing this algorithm, we discuss how to effectively implement a priority queue.

STL `class priority_queue`

The STL provides the `class` template `priority_queue<elemType>`, where the data type of the queue elements is specified by `elemType`. This class template is contained in the STL header file `queue`. You can specify the priority of the elements of a priority queue in various ways. The default priority criteria for the queue elements uses the less-than operator, `<`. For example, a program that implements a priority queue of numbers could use the operator `<` to assign the priority to the numbers so that larger numbers are always at the front of the queue. If you design your own class to implement the queue elements, you can specify your priority rule by overloading the less-than operator, `<`, to compare the elements. You could also define a comparison function to specify the priority. The implementation of comparison functions is discussed in Chapter 13.

Application of Queues: Simulation

A technique in which one system models the behavior of another system is called **simulation**. For example, physical simulators include wind tunnels used to experiment with the design of car bodies and flight simulators are used to train airline pilots. Simulation techniques are used when it is too expensive or dangerous to experiment with real systems. You can also design computer models to study the behavior of real systems. (We will describe some real systems modeled by computers shortly.) Simulating the behavior of an expensive or dangerous experiment using a computer model is usually less expensive than using the real system, and a good way to gain insight without putting human life in danger. Moreover, computer simulations are particularly useful for complex systems where it is difficult to construct a mathematical model. For such systems, computer models can retain descriptive accuracy. In mathematical simulations, the steps of a program are used to model the behavior of a real system. Let us consider one such problem.

The manager of a local movie theater is hearing complaints from customers about the time they have to wait in line to buy tickets. The theater currently has only one cashier. Another theater is preparing to open in the neighborhood and the manager is afraid of losing customers. The manager wants to hire enough cashiers so that a customer does not have to wait too long to buy a ticket, but does not want to hire extra cashiers on a trial basis and potentially waste time and money. One thing that the manager would like to

know is the average time a customer has to wait for service. The manager wants someone to write a program to simulate the behavior of the theater.

In computer simulation, the objects being studied are usually represented as data. For the theater problem, some of the objects are the customers and the cashier. The cashier serves the customers and we want to determine a customer's average waiting time. Actions are implemented by writing algorithms, which in a programming language are implemented with the help of functions. Thus, functions are used to implement the actions of the objects. In C++, we can combine the data and the operations on that data into a single unit with the help of classes. Thus, objects can be represented as classes. The member variables of the class describe the properties of the objects, and the function members describe the actions on that data. This change in simulation results can also occur if we change the values of the data or modify the definitions of the functions (that is, modify the algorithms implementing the actions). The main goal of a computer simulation is to either generate results showing the performance of an existing system or predict the performance of a proposed system.

In the theater problem, when the cashier is serving a customer, the other customers must wait. Because customers are served on a first-come, first-served basis and queues are an effective way to implement a First In First Out system, queues are important data structures for use in computer simulations. This section examines computer simulations in which queues are the basic data structure. These simulations model the behavior of systems, called **queuing systems**, in which queues of objects are waiting to be served by various servers. In other words, a queuing system consists of servers and queues of objects waiting to be served. We deal with a variety of queuing systems on a daily basis. For example, a grocery store and a banking system are both queuing systems. Furthermore, when you send a print request to a networked printer that is shared by many people, your print request goes in a queue. Print requests that arrived before your print request are usually completed before yours. Thus, the printer acts as the server when a queue of documents is waiting to be printed.

Designing a Queuing System

In this section, we describe a queuing system that can be used in a variety of applications, such as a bank, grocery store, movie theater, printer, or mainframe environment in which several people are trying to use the same processors to execute their programs. To describe a queuing system, we use the term **server** for the object that provides the service. For example, in a bank, a teller is a server; in a grocery store or movie theater, a cashier is a server. We will call the object receiving the service the **customer**, and the service time—the time it takes to serve a customer—the **transaction time**.

Because a queuing system consists of servers and a queue of waiting objects, we will model a system that consists of a list of servers and a waiting queue holding the customers to be served. The customer at the front of the queue waits for the next available server. When a server becomes free, the customer at the front of the queue moves to the free server to be served.

When the first customer arrives, all servers are free and the customer moves to the first server. When the next customer arrives, if a server is available, the customer immediately moves to the available server; otherwise, the customer waits in the queue. To model a queuing system, we need to know the number of servers, the expected arrival time of a customer, the time between the arrivals of customers, and the number of events affecting the system.

Let us again consider the movie theater system. The performance of the system depends on how many servers are available, how long it takes to serve a customer, and how often a customer arrives. If it takes too long to serve a customer and customers arrive frequently, then more servers are needed. This system can be modeled as a time-driven simulation. In a **time-driven simulation**, the clock is implemented as a counter and the passage of, say, 1 minute can be implemented by incrementing the counter by 1. The simulation is run for a fixed amount of time. If the simulation needs to be run for 100 minutes, the counter starts at 1 and goes up to 100, which can be implemented by using a loop.

For the simulation described in this section, we want to determine the average wait time for a customer. To calculate the average wait time for a customer, we need to add the waiting time of each customer, and then divide the sum by the number of customers who have arrived. When a customer arrives, he or she goes to the end of the queue and the customer's waiting time starts. If the queue is empty and a server is free, the customer is served immediately and so this customer's waiting time is zero. On the other hand, if when the customer arrives and either the queue is nonempty or all the servers are busy, the customer must wait for the next available server and, therefore, this customer's waiting time starts. We can keep track of the customer's waiting time by using a timer for each customer. When a customer arrives, the timer is set to 0, which is incremented after each clock unit.

Suppose that, on average, it takes five minutes for a server to serve a customer. When a server becomes free and the waiting customer's queue is nonempty, the customer at the front of the queue proceeds to begin the transaction. Thus, we must keep track of the time a customer is with a server. When the customer arrives at a server, the transaction time is set to five and is decremented after each clock unit. When the transaction time becomes zero, the server is marked free. Hence, the two objects needed to implement a time-driven computer simulation of a queuing system are the customer and the server.

Before designing the main algorithm to implement the simulation, we design classes to implement each of the two objects: *customer* and *server*.

Customer

Every customer has a customer number, arrival time, waiting time, transaction time, and departure time. If we know the arrival time, waiting time, and transaction time, we can determine the departure time by adding these three times. Let us call the class to implement the customer object `customerType`. It follows that the `class customerType` has four member variables: the `customerNumber`, `arrivalTime`, `waitingTime`, and `transactionTime`,

each of type `int`. The basic operations that must be performed on an object of type `customerType` are as follows: Set the customer's number, arrival time, and waiting time; increment the waiting time by one clock unit; return the waiting time; return the arrival time; return the transaction time; and return the customer number. The following `class`, `customerType`, implements the customer as an ADT:

```
//*************************************************************
// Author: D.S. Malik
//
// class customerType
// This class specifies the members to implement a customer.
//*************************************************************

class customerType
{
public:
    customerType(int cN = 0, int arrvTime = 0, int wTime = 0,
                 int tTime = 0);
      //Constructor to initialize the instance variables
      //according to the parameters
      //If no value is specified in the object declaration,
      //the default values are assigned.
      //Postcondition: customerNumber = cN; arrivalTime = arrvTime;
      //    waitingTime = wTime; transactionTime = tTime

    void setCustomerInfo(int cN = 0, int inTime = 0,
                         int wTime = 0, int tTime = 0);
      //Function to initialize the instance variables.
      //Instance variables are set according to the parameters.
      //Postcondition: customerNumber = cN; arrivalTime = arrvTime;
      //    waitingTime = wTime; transactionTime = tTime;

    int getWaitingTime() const;
      //Function to return the waiting time of a customer.
      //Postcondition: The value of waitingTime is returned.

    void setWaitingTime(int time);
      //Function to set the waiting time of a customer.
      //Postcondition: waitingTime = time;

    void incrementWaitingTime();
      //Function to increment the waiting time by one time unit.
      //Postcondition: waitingTime++;

    int getArrivalTime() const;
      //Function to return the arrival time of a customer.
      //Postcondition: The value of arrivalTime is returned.

    int getTransactionTime() const;
      //Function to return the transaction time of a customer.
      //Postcondition: The value of transactionTime is returned.
```

```
    int getCustomerNumber() const;
      //Function to return the customer number.
      //Postcondition: The value of customerNumber is returned.

private:
    int customerNumber;
    int arrivalTime;
    int waitingTime;
    int transactionTime;
};
```

Figure 8-11 shows the UML diagram of the `class customerType`.

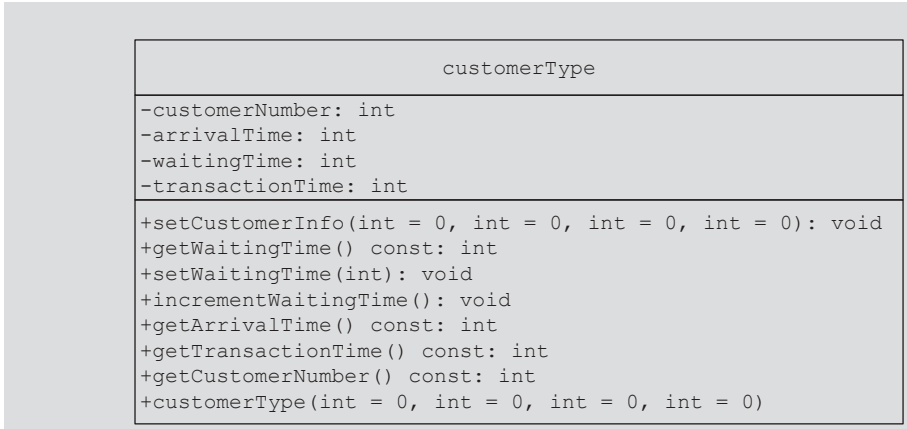


FIGURE 8-11 UML diagram of the `class customerType`

The definitions of the member functions of the `class customerType` follow easily from their descriptions. Next, we give the definitions of the member functions of the `class customerType`.

The function `setCustomerInfo` uses the values of the parameters to initialize `customerNumber`, `arrivalTime`, `waitingTime`, and `transactionTime`. Its definition is as follows:

```
void customerType::setCustomerInfo(int cN, int arrvTime,
                                   int wTime, int tTime)
{
    customerNumber = cN;
    arrivalTime = arrvTime;
    waitingTime = wTime;
    transactionTime = tTime;
}
```

The definition of the constructor is similar to the definition of the function `setCustomerInfo`. It uses the values of the parameters to initialize `customerNumber`,

arrivalTime, waitingTime, and transactionTime. To make debugging easier, we use the function setCustomerInfo to write the definition of the constructor, which is given next.

```
customerType::customerType(int cN, int arrvTime,
                           int wTime, int tTime)
{
    setCustomerInfo(cN, arrvTime, wTime, tTime);
}
```

The function getWaitingTime returns the current waiting time. The definition of the function getWaitingTime is as follows:

```
int customerType::getWaitingTime() const
{
    return waitingTime;
}
```

The function incrementWaitingTime increments the value of waitingTime. Its definition is as follows:

```
void customerType::incrementWaitingTime()
{
    waitingTime++;
}
```

The definitions of the functions setWaitingTime, getArrivalTime, getTransactionTime, and getCustomerNumber are left as an exercise for you, (see Programming Exercise 8 a the end of this chapter).

Server

At any given time unit, the server is either busy serving a customer or is free. We use a string variable to set the status of the server. Every server has a timer and because the program might need to know which customer is served by which server, the server also stores the information of the customer being served. Thus, three member variables are associated with a server: the status, the transactionTime, and the currentCustomer. Some of the basic operations that must be performed on a server are as follows: Check whether the server is free; set the server as free; set the server as busy; set the transaction time (that is, how long it takes to serve the customer); return the remaining transaction time (to determine whether the server should be set to free); if the server is busy after each time unit, decrement the transaction time by one time unit; and so on. The following class, serverType, implements the server as an ADT:

```
//*********************************************************
// Author: D.S. Malik
//
// class serverType
// This class specifies the members to implement a server.
//*********************************************************
```

```
class serverType
{
public:
    serverType();
      //Default constructor
      //Sets the values of the instance variables to their default
      //values.
      //Postcondition: currentCustomer is initialized by its
      //    default constructor; status = "free"; and the
      //    transaction time is initialized to 0.

    bool isFree() const;
      //Function to determine if the server is free.
      //Postcondition: Returns true if the server is free,
      //    otherwise returns false.

    void setBusy();
      //Function to set the status of the server to busy.
      //Postcondition: status = "busy";

    void setFree();
      //Function to set the status of the server to "free."
      //Postcondition: status = "free";

    void setTransactionTime(int t);
      //Function to set the transaction time according to the
      //parameter t.
      //Postcondition: transactionTime = t;

    void setTransactionTime();
      //Function to set the transaction time according to
      //the transaction time of the current customer.
      //Postcondition:
      //   transactionTime = currentCustomer.transactionTime;

    int getRemainingTransactionTime() const;
      //Function to return the remaining transaction time.
      //Postcondition: The value of transactionTime is returned.

    void decreaseTransactionTime();
      //Function to decrease the transactionTime by 1 unit.
      //Postcondition: transactionTime--;

    void setCurrentCustomer(customerType cCustomer);
      //Function to set the info of the current customer
      //according to the parameter cCustomer.
      //Postcondition: currentCustomer = cCustomer;

    int getCurrentCustomerNumber() const;
      //Function to return the customer number of the current
      //customer.
      //Postcondition: The value of customerNumber of the
      //    current customer is returned.
```

```
    int getCurrentCustomerArrivalTime() const;
      //Function to return the arrival time of the current
      //customer.
      //Postcondition: The value of arrivalTime of the current
      //    customer is returned.

    int getCurrentCustomerWaitingTime() const;
      //Function to return the current waiting time of the
      //current customer.
      //Postcondition: The value of transactionTime is returned.

    int getCurrentCustomerTransactionTime() const;
      //Function to return the transaction time of the
      //current customer.
      //Postcondition: The value of transactionTime of the
      //    current customer is returned.

private:
    customerType currentCustomer;
    string status;
    int transactionTime;
};
```

Figure 8-12 shows the UML diagram of the **class serverType**.

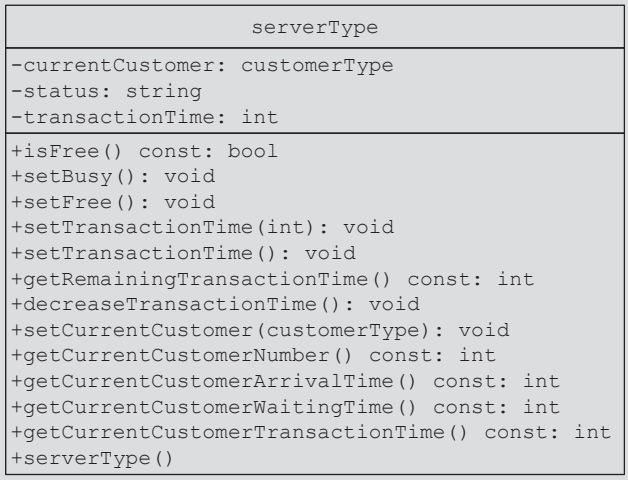


FIGURE 8-12 UML diagram of the `class serverType`

The definitions of some of the member functions of the **class serverType** are as follows:

```
serverType::serverType()
{
    status = "free";
    transactionTime = 0;
}

bool serverType::isFree() const
{
    return (status == "free");
}

void serverType::setBusy()
{
    status = "busy";
}

void serverType::setFree()
{
    status = "free";
}

void serverType::setTransactionTime(int t)
{
    transactionTime = t;
}

void serverType::setTransactionTime()
{
    int time;

    time = currentCustomer.getTransactionTime();

    transactionTime = time;
}

void serverType::decreaseTransactionTime()
{
    transactionTime--;
}
```

We leave the definitions of the functions `getRemainingTransactionTime`, `setCurrentCustomer`, `getCurrentCustomerNumber`, `getCurrentCustomerArrivalTime`, `getCurrentCustomerWaitingTime`, and `getCurrentCustomerTransactionTime` as an exercise for you, (see Programming Exercise 8 at the end of this chapter).

Because we are designing a simulation program that can be used in a variety of applications, we need to design two more classes: a class to create and process a list of servers and a class to create and process a queue of waiting customers. The next two sections describe each of these classes.

Server List

A server list is a set of servers. At any given time, a server is either free or busy. For the customer at the front of the queue, we need to find a server in the list that is free. If all the servers are busy, the customer must wait until one of the servers becomes free. Thus, the class that implements a list of servers has two member variables: one to store the number of servers and one to maintain a list of servers. Using dynamic arrays, depending on the number of servers specified by the user, a list of servers is created during program execution. Some of the operations that must be performed on a server list are as follows: Return the server number of a free server; when a customer gets ready to do business and a server is available, set the server to busy; when the simulation ends, some of the servers might still be busy, so return the number of busy servers; after each time unit, reduce the `transactionTime` of each busy server by one time unit; and if the `transactionTime` of a server becomes zero, set the server to free. The following `class`, `serverListType`, implements the list of servers as an ADT:

```
//*************************************************************
// Author: D.S. Malik
//
// class serverListType
// This class specifies the members to implement a list of
// servers.
//*************************************************************

class serverListType
{
public:
    serverListType(int num = 1);
      //Constructor to initialize a list of servers
      //Postcondition: numOfServers = num
      //    A list of servers, specified by num, is created and
      //    each server is initialized to "free".

    ~serverListType();
      //Destructor
      //Postcondition: The list of servers is destroyed.

    int getFreeServerID() const;
      //Function to search the list of servers.
      //Postcondition: If a free server is found, returns its ID;
      //    otherwise, returns -1.

    int getNumberOfBusyServers() const;
      //Function to return the number of busy servers.
      //Postcondition: The number of busy servers is returned.

    void setServerBusy(int serverID, customerType cCustomer,
                       int tTime);
      //Function to set a server busy.
```

```
      //Postcondition: The server specified by serverID is set to
      //    "busy", to serve the customer specified by cCustomer,
      //    and the transaction time is set according to the
      //    parameter tTime.

    void setServerBusy(int serverID, customerType cCustomer);
      //Function to set a server busy.
      //Postcondition: The server specified by serverID is set to
      //    "busy", to serve the customer specified by cCustomer.

    void updateServers(ostream& outFile);
      //Function to update the status of a server.
      //Postcondition: The transaction time of each busy server
      //    is decremented by one unit. If the transaction time of
      //    a busy server is reduced to zero, the server is set to
      //    "free". Moreover, if the actual parameter corresponding
      //    to outFile is cout, a message indicating which customer
      //    has been served is printed on the screen, together with the
      //    customer's departing time. Otherwise, the output is sent
      //    to a file specified by the user.

private:
    int numOfServers;
    serverType *servers;
};
```

Figure 8-13 shows the UML diagram of the `class serverListType`.

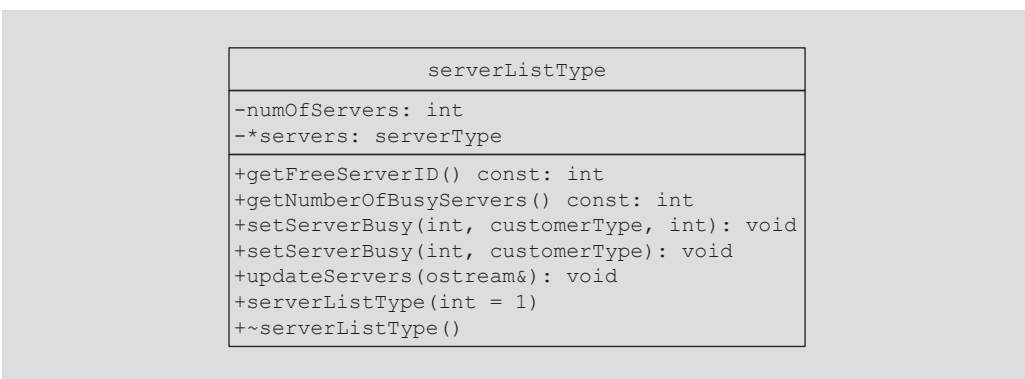


FIGURE 8-13 UML diagram of the `class serverListType`

Following are the definitions of the member functions of the `class serverListType`. The definitions of the constructor and destructor are straightforward.

```
serverListType::serverListType(int num)
{
    numOfServers = num;
    servers = new serverType[num];
}
```

```
serverListType::~serverListType()
{
    delete [] servers;
}
```

The function `getFreeServerID` searches the list of servers. If a free server is found, it returns the server's ID; otherwise, the value `-1` is returned, which indicates that all the servers are busy. The definition of this function is as follows:

```
int serverListType::getFreeServerID() const
{
    int serverID = -1;

    for (int i = 0; i < numOfServers; i++)
        if (servers[i].isFree())
        {
            serverID = i;
            break;
        }

    return serverID;
}
```

The function `getNumberOfBusyServers` searches the list of servers and determines the number of busy servers. The number of busy servers is returned. The definition of this function is as follows:


```
int serverListType::getNumberOfBusyServers() const
{
    int busyServers = 0;

    for (int i = 0; i < numOfServers; i++)
        if (!servers[i].isFree())
            busyServers++;

    return busyServers;
}
```

The function `setServerBusy` sets a server to busy. This function is overloaded. The `serverID` of the server that is set to busy is passed as a parameter to this function. One function sets the server's transaction time according to the parameter `tTime`; the other function sets it by using the transaction time stored in the object `cCustomer`. The transaction time is later needed to determine the average waiting time. The definitions of these functions are as follows:

```
void serverListType::setServerBusy(int serverID,
                                   customerType cCustomer, int tTime)
{
    servers[serverID].setBusy();
    servers[serverID].setTransactionTime(tTime);
    servers[serverID].setCurrentCustomer(cCustomer);
}
```

```
void serverListType::setServerBusy(int serverID,
                                   customerType cCustomer)
{
    int time = cCustomer.getTransactionTime();

    servers[serverID].setBusy();
    servers[serverID].setTransactionTime(time);
    servers[serverID].setCurrentCustomer(cCustomer);
}
```

The definition of the function `updateServers` is quite straightforward. Starting at the first server, it searches the list of servers for busy servers. When a busy server is found, its `transactionTime` is decremented by 1. If the `transactionTime` reduces to zero, the server is set to free. If the `transactionTime` of a busy server reduces to zero, the transaction of the customer being served by the server is complete. If the actual parameter corresponding to `outFile` is `cout`, a message indicating which customer has been served is printed on the screen, together with the customer's departing time. Otherwise, the output is sent to a file specified by the user. The definition of this function is as follows:

```
void serverListType::updateServers(ostream& outF)
{
   for (int i = 0; i < numOfServers; i++)
       if (!servers[i].isFree())
       {
          servers[i].decreaseTransactionTime();

          if (servers[i].getRemainingTransactionTime() == 0)
          {
              outF << "From server number " << (i + 1)
                   << " customer number "
                   << servers[i].getCurrentCustomerNumber()
                   << "\n     departed at clock unit "
                   << servers[i].getCurrentCustomerArrivalTime()
                   + servers[i].getCurrentCustomerWaitingTime()
                   + servers[i].getCurrentCustomerTransactionTime()
                   << endl;
              servers[i].setFree();
          }
       }
}
```

Waiting Customers Queue

When a customer arrives, he or she goes to the end of the queue. When a server becomes available, the customer at the front of the queue leaves to conduct the transaction. After each time unit, the waiting time of each customer in the queue is incremented by 1. The ADT `queueType` designed in this chapter has all the operations needed to implement a queue, except the operation of incrementing the waiting time of each customer in the queue by one time unit. We will derive a `class`, `waitingCustomerQueueType`, from the `class queueType` and add the additional operations to implement the customer queue. The definition of the `class waitingCustomerQueueType` is as follows:

```
//*************************************************************
// Author: D.S. Malik
//
// class waitingCustomerQueueType
// This class extends the class queueType to implement a list
// of waiting customers.
//*************************************************************

class waitingCustomerQueueType: public queueType<customerType>
{
public:
    waitingCustomerQueueType(int size = 100);
      //Constructor
      //Postcondition: The queue is initialized according to the
      //    parameter size. The value of size is passed to the
      //    constructor of queueType.

    void updateWaitingQueue();
      //Function to increment the waiting time of each
      //customer in the queue by one time unit.
};
```

NOTE

Notice that the `class waitingCustomerQueueType` is derived from the `class queueType`, which implements the queue in an array. You can also derive it from the `class linkedQueueType`, which implements the queue in a linked list. We leave the details as an exercise for you.

The definitions of the member functions are given next. The definition of the constructor is as follows:

```
waitingCustomerQueueType::waitingCustomerQueueType(int size)
                          :queueType<customerType>(size)
{
}
```

The function `updateWaitingQueue` increments the waiting time of each customer in the queue by one time unit. The `class waitingCustomerQueueType` is derived from the `class queueType`. Because the member variables of `queueType` are `private`, the function `updateWaitingQueue` cannot directly access the elements of the queue. The only way to access the elements of the queue is to use the `deleteQueue` operation. After incrementing the waiting time, the element can be put back into the queue by using the `addQueue` operation.

The `addQueue` operation inserts the element at the end of the queue. If we perform the `deleteQueue` operation followed by the `addQueue` operation for each element of the queue, eventually the front element again becomes the front element. Given that each `deleteQueue` operation is followed by an `addQueue` operation, how do we determine that all the elements of the queue have been processed? We cannot use the `isEmptyQueue` or `isFullQueue` operations on the queue because the queue will never be empty or full.

One solution to this problem is to create a temporary queue. Every element of the original queue is removed, processed, and inserted into the temporary queue. When the original queue becomes empty, all of the elements in the queue are processed. We can then copy the elements from the temporary queue back into the original queue. However, this solution requires us to use extra memory space, which could be significant. Also, if the queue is large, extra computer time is needed to copy the elements from the temporary queue back into the original queue. Let us look into another solution.

In the second solution, before starting to update the elements of the queue, we can insert a dummy customer with a waiting time of, say `-1`. During the update process, when we arrive at the customer with the waiting time of `-1`, we can stop the update process without processing the customer with the waiting time of `-1`. If we do not process the customer with the waiting time `-1`, this customer is removed from the queue and after processing all the elements of the queue, the queue will contain no extra elements. This solution does not require us to create a temporary queue, so we do not need extra computer time to copy the elements back into the original queue. We will use this solution to update the queue. Therefore, the definition of the function `updateWaitingQueue` is as follows:

```
void waitingCustomerQueueType::updateWaitingQueue()
{
    customerType cust;

    cust.setWaitingTime(-1);
    int wTime = 0;

    addQueue(cust);

    while (wTime != -1)
    {
        cust = front();
        deleteQueue();

        wTime = cust.getWaitingTime();
        if (wTime == -1)
            break;
        cust.incrementWaitingTime();
        addQueue(cust);
    }
}
```

Main Program

To run the simulation, we first need to get the following information:

- The number of time units the simulation should run. Assume that each time unit is one minute.
- The number of servers.
- The amount of time it takes to serve a customer—that is, the transaction time.
- The approximate time between customer arrivals.

These pieces of information are called simulation parameters. By changing the values of these parameters, we can observe the changes in the performance of the system. We can write a function, `setSimulationParameters`, to prompt the user to specify these values. The definition of this function is as follows:

```
void setSimulationParameters(int& sTime, int& numOfServers,
                             int& transTime, int& tBetweenCArrival)
{
    cout << "Enter the simulation time: ";
    cin >> sTime;
    cout << endl;

    cout << "Enter the number of servers: ";
    cin >> numOfServers;
    cout << endl;

    cout << "Enter the transaction time: ";
    cin >> transTime;
    cout << endl;

    cout << "Enter the time between customers arrival: ";
    cin >> tBetweenCArrival;
    cout << endl;
}
```


When a server becomes free and the customer queue is nonempty, we can move the customer at the front of the queue to the free server to be served. Moreover, when a customer starts the transaction, the waiting time ends. The waiting time of the customer is added to the total waiting time. The general algorithm to start the transaction (supposing that `serverID` denotes the ID of the free server) is as follows:

1. Remove the customer from the front of the queue.

   ```
   customer = customerQueue.front();
   customerQueue.deleteQueue();
   ```

2. Update the total waiting time by adding the current customer's waiting time to the previous total waiting time.

   ```
   totalWait = totalWait + customer.getWaitingTime();
   ```

3. Set the free server to begin the transaction.

   ```
   serverList.setServerBusy(serverID, customer, transTime);
   ```

To run the simulation, we need to know the number of customers arriving at a given time unit and how long it takes to serve the customer. We use the Poisson distribution from statistics, which says that the probability of y events occurring at a given time is given by the formula:

$$P(y) = \frac{\lambda^y e^{-\lambda}}{y!}, y = 0, 1, 2, \ldots,$$

where λ is the expected value that y events occur at that time. Suppose that, on average, a customer arrives every four minutes. During this four-minute period, the customer can arrive at any one of the four minutes. Assuming an equal likelihood of each of the four minutes, the expected value that a customer arrives in each of the four minutes is, therefore, $1 / 4 = 0.25$. Next, we need to determine whether the customer actually arrives at a given minute.

Now $P(0) = e^{-\lambda}$ is the probability that no event occurs at a given time. One of the basic assumptions of the Poisson distribution is that the probability of more than one outcome occurring in a short time interval is negligible. For simplicity, we assume that only one customer arrives at a given time unit. Thus, we use $e^{-\lambda}$ as the cutoff point to determine whether a customer arrives at a given time unit. Suppose that, on average, a customer arrives every four minutes. Then $\lambda = 0.25$. We can use an algorithm to generate a number between 0 and 1. If the value of the number generated is $> e^{-0.25}$, we can assume that the customer arrived at a particular time unit. For example, suppose that *rNum* is a random number such that $0 \leq rNum \leq 1$. If $rNum > e^{-0.25}$, the customer arrived at the given time unit.

We now describe the function `runSimulation` to implement the simulation. Suppose that we run the simulation for 100 time units and customers arrive at time units 93, 96, and 100. The average transaction time is 5 minutes—that is, 5 time units. For simplicity, assume that we have only one server and the server becomes free at time unit 97, and that all customers arriving before time unit 93 have been served. When the server becomes free at time unit 97, the customer arriving at time unit 93 starts the transaction. Because the transaction of the customer arriving at time unit 93 starts at time unit 97 and it takes 5 minutes to complete a transaction, when the simulation loop ends, the customer arriving at time unit 93 is still at the server. Moreover, customers arriving at time units 96 and 100 are in the queue. For simplicity, we assume that when the simulation loop ends, the customers at the servers are considered served. The general algorithm for this function is as follows:

1. Declare and initialize the variables such as the simulation parameters, customer number, clock, total and average waiting times, number of customers arrived, number of customers served, number of customers left in the waiting queue, number of customers left with the servers, `waitingCustomersQueue`, and a list of servers.
2. The main loop is as follows:

```
for (clock = 1; clock <= simulationTime; clock++)
{
```

   2.1. Update the server list to decrement the transaction time of each busy server by one time unit.

   2.2. If the customer's queue is nonempty, increment the waiting time of each customer by one time unit.

   2.3. If a customer arrives, increment the number of customers by 1 and add the new customer to the queue.

   2.4. If a server is free and the customer's queue is nonempty, remove a customer from the front of the queue and send the customer to the free server.

```
}
```

3. Print the appropriate results. Your results must include the number of customers left in the queue, the number of customers still with servers, the number of customers arrived, and the number of customers who actually completed a transaction.

Once you have designed the function `runSimulation`, the definition of the function `main` is simple and straightforward because the function `main` calls only the function `runSimulation`. (See Programming Exercise 8 at the end of this chapter.)

When we tested our version of the simulation program, we generated the following results. We assumed that the average transaction time is 5 minutes and that on average a customer arrives every 4 minutes, and we used a random number generator to generate a number between `0` and `1` to decide whether a customer arrived at a given time unit.

Sample Run:

```
Customer number 1 arrived at time unit 4
Customer number 2 arrived at time unit 8
From server number 1 customer number 1
     departed at clock unit 9
Customer number 3 arrived at time unit 9
Customer number 4 arrived at time unit 12
From server number 1 customer number 2
     departed at clock unit 14
From server number 1 customer number 3
     departed at clock unit 19
Customer number 5 arrived at time unit 21
From server number 1 customer number 4
     departed at clock unit 24
From server number 1 customer number 5
     departed at clock unit 29
Customer number 6 arrived at time unit 37
Customer number 7 arrived at time unit 38
Customer number 8 arrived at time unit 41
From server number 1 customer number 6
     departed at clock unit 42
Customer number 9 arrived at time unit 43
Customer number 10 arrived at time unit 44
From server number 1 customer number 7
     departed at clock unit 47
Customer number 11 arrived at time unit 49
Customer number 12 arrived at time unit 51
From server number 1 customer number 8
     departed at clock unit 52
Customer number 13 arrived at time unit 52
Customer number 14 arrived at time unit 53
Customer number 15 arrived at time unit 54
From server number 1 customer number 9
     departed at clock unit 57
Customer number 16 arrived at time unit 59
From server number 1 customer number 10
     departed at clock unit 62
```

```
Customer number 17 arrived at time unit 66
From server number 1 customer number 11
     departed at clock unit 67
Customer number 18 arrived at time unit 71
From server number 1 customer number 12
     departed at clock unit 72
From server number 1 customer number 13
     departed at clock unit 77
Customer number 19 arrived at time unit 78
From server number 1 customer number 14
     departed at clock unit 82
From server number 1 customer number 15
     departed at clock unit 87
Customer number 20 arrived at time unit 90
From server number 1 customer number 16
     departed at clock unit 92
Customer number 21 arrived at time unit 92
From server number 1 customer number 17
     departed at clock unit 97

The simulation ran for 100 time units
Number of servers: 1
Average transaction time: 5
Average arrival time difference between customers: 4
Total waiting time: 269
Number of customers that completed a transaction: 17
Number of customers left in the servers: 1
The number of customers left in queue: 3
Average waiting time: 12.81
************** END SIMULATION *************
```

QUICK REVIEW

1. A queue is a data structure in which the items are added at one end and removed from the other end.
2. A queue is a First In First Out (FIFO) data structure.
3. The basic operations on a queue are as follows: Add an item to the queue, remove an item from the queue, retrieve the first and last element of the queue, initialize the queue, check whether the queue is empty, and check whether the queue is full.
4. A queue can be implemented as an array or a linked list.
5. The middle elements of a queue should not be accessed directly.
6. If the queue is nonempty, the function `front` returns the front element of the queue and the function `back` returns the last element in the queue.
7. Queues are restricted versions of arrays and linked lists.

EXERCISES

1. Consider the following statements:

```
queueType<int> queue;
int x, y;
```

Show what is output by the following segment of code:

```
x = 4;
y = 5;
queue.addQueue(x);
queue.addQueue(y);
x = queue.front();
queue.deleteQueue();
queue.addQueue(x + 5);
queue.addQueue(16);
queue.addQueue(x);
queue.addQueue(y - 3);

cout << "Queue Elements: ";
while (!queue.isEmptyQueue())
{
    cout << queue.front() << " ";
    queue.deleteQueue();
}
cout << endl;
```

2. Consider the following statements:

```
stackType<int> stack;
queueType<int> queue;
int x;
```

Suppose the input is:

```
15 28 14 22 64 35 19 32 7 11 13 30 -999
```

Show what is written by the following segment of code:

```
stack.push(0);
queue.addQueue(0);
cin >> x;

while (x != -999)
{
    switch (x % 4)
    {
    case 0:
        stack.push(x);
        break;
    case 1:
        if (!stack.isEmptyStack())
        {
            cout << "Stack Element = " << stack.top()
                 << endl;
            stack.pop();
        }
```

```
            else
                cout << "Sorry, the stack is empty." << endl;
                break;
        case 2:
            queue.addQueue(x);
            break;
        case 3:
            if (!queue.isEmptyQueue())
            {
                cout << "Queue Element = " << queue.front()
                     << endl;
                queue.deleteQueue();
            }
            else
                cout << "Sorry, the queue is empty." << endl;
            break;
        } //end switch

        cin >> x;
    } //end while

    cout << "Stack Elements: ";
    while (!stack.isEmptyStack())
    {
        cout << stack.top() << " ";
        stack.pop();
    }

    cout << endl;

    cout << "Queue Elements: ";
    while (!queue.isEmptyQueue())
    {
        cout << queue.front() << " ";
        queue.deleteQueue();
    }
    cout << endl;
```

3. What does the following function do?

```
void mystery(queueType<int>& q)
{
    stackType<int> s;

    while (!q.isEmptyQueue())
    {
        s.push(q.front());
        q.deleteQueue();
    }

    while (!s.isEmptyStack())
    {
        q.addQueue(2 * s.top());
        s.pop();
    }
}
```

4. What is the effect of the following statements? If a statement is invalid, explain why it is invalid. The classes `queueADT`, `queueType`, and `linkedQueueType` are as defined in this chapter.

   a. `queueADT<int> newQueue;`

   b. `queueType <double> sales(-10);`

   c. `queueType <string> names;`

   d. `linkedQueueType <int> numQueue(50);`

5. What is the output of the following program segment?

```
linkedQueueType<int> queue;

queue.addQueue(10);
queue.addQueue(20);
cout << queue.front() << endl;
queue.deleteQueue();
queue.addQueue(2 * queue.back());
queue.addQueue(queue.front());
queue.addQueue(5);
queue.addQueue(queue.back() - 2);

linkedQueueType<int> tempQueue;

tempQueue = queue;

while (!tempQueue.isEmptyQueue())
{
    cout << tempQueue.front() << " ";
    tempQueue.deleteQueue();
}

cout << endl;

cout << queue.front() << " " << queue.back() << endl;
```

6. Suppose that `queue` is a `queueType` object and the size of the array implementing queue is `100`. Also, suppose that the value of `queueFront` is `50` and the value of `queueRear` is `99`.

   a. What are the values of `queueFront` and `queueRear` after adding an element to `queue`?

   b. What are the values of `queueFront` and `queueRear` after removing an element from `queue`?

7. Suppose that `queue` is a `queueType` object and the size of the array implementing queue is `100`. Also, suppose that the value of `queueFront` is `99` and the value of `queueRear` is `25`.

a. What are the values of `queueFront` and `queueRear` after adding an element to `queue`?

b. What are the values of `queueFront` and `queueRear` after removing an element from `queue`?

8. Suppose that `queue` is a `queueType` object and the size of the array implementing queue is 100. Also, suppose that the value of `queueFront` is `25` and the value of `queueRear` is `75`.

   a. What are the values of `queueFront` and `queueRear` after adding an element to `queue`?

   b. What are the values of `queueFront` and `queueRear` after removing an element from `queue`?

9. Suppose that `queue` is a `queueType` object and the size of the array implementing queue is `100`. Also, suppose that the value of `queueFront` is `99` and the value of `queueRear` is `99`.

   a. What are the values of `queueFront` and `queueRear` after adding an element to `queue`?

   b. What are the values of `queueFront` and `queueRear` after removing an element from `queue`?

10. Suppose that `queue` is implemented as an array with the special reserved slot, as described in this chapter. Also, suppose that the size of the array implementing `queue` is `100`. If the value of `queueFront` is `50`, what is the position of the first queue element?

11. Suppose that `queue` is implemented as an array with the special reserved slot, as described in this chapter. Suppose that the size of the array implementing `queue` is `100`. Also, suppose that the value of `queueFront` is `74` and the value of `queueRear` is `99`.

    a. What are the values of `queueFront` and `queueRear` after adding an element to `queue`?

    b. What are the values of `queueFront` and `queueRear` after removing an element from `queue`? Also, what is the position of the removed queue element?

12. Write a function template, `reverseQueue`, that takes as a parameter a queue object and uses a stack object to reverse the elements of the queue.

13. Add the operation `queueCount` to the `class queueType` (the array implementation of queues), which returns the number of elements in the queue. Write the definition of the function template to implement this operation.

14. Draw the UML diagram of the `class queueADT`.

15. Draw the UML diagram of the `class queueType`.

16. Draw the UML diagram of the `class linkedQueueType`.

PROGRAMMING EXERCISES

1. Write the definitions of the functions to overload the assignment operator and copy constructor for the `class queueType`. Also, write a program to test these operations.
2. Write the definitions of the functions to overload the assignment operator and copy constructor for the `class linkedQueueType`. Also, write a program to test these operations.
3. This chapter described the array implementation of queues that use a special array slot, called the reserved slot, to distinguish between an empty and a full queue. Write the definition of the class and the definitions of the function members of this queue design. Also, write a test program to test various operations on a queue.
4. Write the definition of the function `moveNthFront` that takes as a parameter a positive integer, *n*. The function moves the *n*th element of the queue to the front. The order of the remaining elements remains unchanged. For example, suppose

   `queue = {5, 11, 34, 67, 43, 55} and` *n* `= 3.`

   After a call to the function `moveNthFront`,

   `queue = {34, 5, 11, 67, 43, 55}.`

   Add this function to the `class queueType`. Also write a program to test your method.
5. Write a program that reads a line of text, changes each uppercase letter to lowercase, and places each letter both in a queue and onto a stack. The program should then verify whether the line of text is a palindrome (a set of letters or numbers that is the same whether read forward or backward).
6. The implementation of a queue in an array, as given in this chapter, uses the variable `count` to determine whether the queue is empty or full. You can also use the variable `count` to return the number of elements in the `queue`. (See Exercise 13.) On the other hand, `class linkedQueueType` does not use such a variable to keep track of the number of elements in the queue. Redefine the `class linkedQueueType` by addding the variable `count` to keep track of the number of elements in the queue. Modify the definitions of the functions `addQueue` and `deleteQueue` as necessary. Add the function `queueCount` to return the number of elements in the queue. Also, write a program to test various operations of the class you defined.
7. Write the definition of the `class linkedQueueType`, which is derived from the `class unorderedLinkedList`, as explained in this chapter. Also write a program to test various operations of this class.

8. a. Write the definitions of the functions `setWaitingTime`, `getArrivalTime`, `getTransactionTime`, and `getCustomerNumber` of the `class customerType` defined in the section, "Application of Queues: Simulation."

   b. Write the definitions of the functions `getRemainingTransactionTime`, `setCurrentCustomer`, `getCurrentCustomerNumber`, `getCurrentCustomerArrivalTime`, `getCurrentCustomerWaitingTime`, and `getCurrentCustomerTransactionTime` of the `class serverType` defined in the section, "Application of Queues: Simulation."

   c. Write the definition of the function `runSimulation` to complete the design of the computer simulation program (see the section, "Application of Queues: Simulation"). Test run your program for a variety of data. Moreover, use a random number generator to decide whether a customer arrived at a given time unit.

9. Redo the simulation program of this chapter so that it uses the STL `class queue` to maintain the list of waiting customers.